Question 1: Consider a FICO pipeline with 4 stages:

IF DISP EX* SU

EX takes 2 (pipelined) cycles for LW, and 1 cycle for ADD. There is only one ADD unit and one LW unit. Two instructions per cycle are fetched by IF, and the DISP queue is unlimited in length.

Show the pipeline timing diagram for the following sequence of instructions as shown in class. You can assume that when as instruction completes and clears the busy bit on its destination register, any instruction, waiting in DISP for that register can fire in the same cycle.

A. LW R1, 0(R2)
B. ADD R3, R1, R2
C. ADD R5, R4, R2
D. LW R6, 0(R4)
E. ADD R7, R1, R5
F. ADD R8, R6, R3

resource conflict-induced stall

		0	1	2	3	4	5	6	7	8
LW	A	IF	DISP r1	EX r1	EX #1	SU				
ADD	B	IF	DISP	DISP	DISP r2	EX #3	SU			
ADD	C		IF	DISP	DISP	DISP 5	EX 5	SU		
LW	D		IF	DISP	DISP	DISP 6	EX 6	EX 6	SU	
ADD	E			IF	DISP	DISP	DISP 7	EX 7	SU	
ADD	F			IF	DISP	DISP	DISP	DISP 8	EX #8	SU
		9	**10**	**11**	**12**	**13**	**14**	**15**	**16**	**17**
	A									
	B									
	C									
	D									
	E									
	F									

- **Highlight boxes where an instruction cannot make forward progress due to resource limitations.**

Question 2: The sequence in question 1 is now executed on an out-of-order processor that has six LW units and six ADD units. As before, LW is pipelined into 2 cycles, ADD is a single-cycle operation. The fetch rate is now 6 instructions/cycle. **Show the contents of the scheduling queue for traditional Tomasulo (no RAT/Pregs) and the register file** at two snapshots in time:

a) After all 6 instructions have been dispatched, and
b) After instructions A, C and D have completed.

Assume tags begin at the value 100. The sizes of CDBs and Scheduling Queue are unlimited. After an instruction completes and goes into State Update, we will update the RF of its destination register and any instructions waiting in SCHED for that register can fire in the same cycle. **Use the given tables for your final answer**. Note: You do not need to show the register values, just the tag, ready and register number fields of the scheduling queue.

a) After all 6 instructions have been dispatched

Scheduling Queue

	FU	Dest Reg	Dest Reg Tag	Src1 Ready	Src1 Tag	Src2 Ready	Src2 Tag
A	LW	R1	100	1		R2 1	
B	ADD	R3	101	R1 0	100	R4 1	
C	ADD	R5	102	R4 1		R2 1	
D	LW	R6	103	1		R4 1	
E	ADD	R7	104	R1 0	100	R5 0	102
F	ADD	R8	105	R6 0	103	R3 0	101

Register File

Register #	Ready	Tag
R1	~~1~~ 0	100
R2	1	
R3	+ 0	~~100~~ 101
R4	1	
R5	+ 0	~~100~~ 102
R6	+ 0	103
R7	+ 0	104

R8	0	105
R9	1	
R10	1	

b) After instructions A, C and D have completed.

	FU	Dest Reg	Dest Reg Tag	Src1 Ready	Src1 Tag	Src2 Ready	Src2 Tag
B	ADD	R3	101	R1 ~~0~~ 1	100	R4 1	N/A
E	ADD	R7	104	R1 ~~0~~ 1	100	R5 ~~0~~ 1	102
F	ADD	R8	105	R6 ~~0~~ 1	103	R3 0	101

Register #	Ready	Tag
R1	~~0~~ 1	100
R2	1	
R3	0	101
R4	1	
R5	~~0~~ 1	102
R6	~~0~~ 1	103
R7	0	104
R8	0	105
R9	1	
R10	1	

↑ before ACD completion

Question 3: The following instruction stream is executed on a superscalar pipeline that uses the PReg/RAT approach for register renaming. Show the state of the scheduling queue, register file and RAT after all the instructions have been dispatched:

A. LW R1, 0(R2)
B. ADD R3, R1, R2
C. ADD R5, R4, R2
D. LW R6, 0(R4)
E. ADD R7, R1, R5
F. ADD R8, R6, R3

You can assume that Pregs start from 0.

Scheduling Queue

inst	FU	dest preg #	src1 preg #	src 2 preg #
A	LW	R1 P8		R2 P1
B	ADD	R3 P9	R1 P8	R2 P1
C	ADD	R5 P10	R4 P3	R2 P1
D	LW	R6 P11		R4 P3
E	ADD	R7 P12	R1 P8	R5 P10
F	ADD	R8 P13	R6 P11	R3 P9

RAT

Areg	Preg
R1	~~P0~~ P8
R2	P1
R3	~~P2~~ P9
R4	P3
R5	~~P4~~ P10
R6	~~P5~~ P11
R7	~~P6~~ P12
R8	~~P7~~ P13

Register File

Preg #	ready
P0	1
P1	1
P2	1
P3	1
P4	1
P5	1
P6	1
P7	1
P8	~~1~~ 0
P9	~~1~~ 0
P10	~~1~~ 0
P11	~~1~~ 0
P12	~~1~~ 0
P13	~~1~~ 0
P14	1
P15	1